



SQLI INCIDENT RESPONSE



SEPTEMBER 1, 2025
AYDEN ROMULUS

Table of Contents

Introduction	2
Part A – Attack Analysis	2
What is an SQL Injection attack?	2
How does it work technically?.....	3
How might this attack have occurred in the patient portal?.....	3
What misconfigurations made it possible?	4
No parameterized queries / prepared statements.....	4
Insufficient server-side input validation (no allowlists)	4
Over-privileged application DB account	5
Why did existing defenses not catch it?.....	5
Part B – The Playbook	5
What is an Incident Response Playbook?.....	5
Incident Response Steps (SQLi Attack Playbook)	6
Flowchart Diagram	7
Tools & Resources.....	9
Web Application Firewall (WAF)	9
Intrusion Detection & Prevention System (IDS/IPS).....	9
Database Activity Monitoring (DAM) Tool	9
Security Information and Event Management (SIEM) System.....	10
Endpoint Detection & Response (EDR) Solution.....	10
Lessons Learned.....	10
Employee Awareness and Training.....	10
Technical Defenses.....	10
Incident Response Improvements	11
Compliance and Monitoring	11
Policy and Governance.....	11
References.....	12

Introduction

This document investigates a hypothetical SQL injection breach in a healthcare system and will develop an incident response playbook to demonstrate the vulnerabilities, response steps and preventative strategies used associated with this type of breach.

Within this scenario, the network of the healthcare organization hosts web-based patient portals, internal hospital management systems, and third-party laboratory interfaces. Personally identifiable information (PII) of over 5,000 patients has been breached and exfiltrated via a web application vulnerability by performing an SQL Injection attack.

Part A – Attack Analysis

What is an SQL Injection attack?

Structured Query Language (SQL) is a standard programming language for managing and manipulating data in a relational database. Information is stored in a structured tabular format (rows and columns) which allows users to perform a variety of tasks like creating complex queries (Amazon AWS, 2024; w3schools, 2019).

An SQL Injection (SQLi) attack is a cyberattack technique where a threat actor inserts malicious SQL code into input fields of a web application. By altering the SQL queries, the application makes to the database, the malicious code gives the attacker the ability to access, modify or remove data without authorization (Hofesh, 2022; OWASP, 2024).

The attack exploits the application's handling of user input, particularly when inputs are not being properly validated, allowing the attacker to alter the intended SQL commands executed by the database (SentinelOne, 2024).

How does it work technically?

Threat actors can craft malicious input into vulnerable input fields, such as form fields, URL parameters or cookies that are used directly in SQL queries. SQL allows for the inclusion of logical conditions like “OR {condition}” in queries which is used by threat actors to trick the database into revealing information.

Attackers may also use comment syntax (e.g., --) to ignore the rest of the original query and ensure that injected code executes.

A simple example is when a query is dynamically built as:

```
SELECT * FROM users WHERE username = 'input_username';
```

If the attacker submits input_username as ‘ OR 1=1 --, the resulting query becomes:

```
SELECT * FROM users WHERE username = ' OR 1=1 --';
```

The condition causes the “WHERE” clause to always be evaluated as true, potentially giving the attacker access to all user records (SentinelOne, 2024; VanMSFT, 2024).

How might this attack have occurred in the patient portal?

This attack might have occurred in a common place such as a “patient search or login/view personal records” field that accepts text or an ID. Suppose the backend of the application builds a query by including the user input.

Because “user_input” is injected directly into the SQL string, an attacker can supply crafted text, so the database interprets it as code, not data (e.g., closing the string, adding an OR clause, UNION, or comment). Once the portal executes the resulting statement, the attacker can:

- Bypass logic (e.g., '1' = '1') to broaden results.
- Enumerate schema to discover PII tables/columns.
- Extract data at scale using “UNION SELECT”/Boolean or time-based blind techniques when error messages are hidden.

To address the outbound traffic, when an attack like this occurs, the web application returns the stolen rows in HTTP responses to the attacker’s client which explains the unusual traffic.

What misconfigurations made it possible?

No parameterized queries / prepared statements

Concatenating input into SQL strings allows user input to change the query structure.

Using placeholders (?, \$1) or Object-Relational Mapping parameter binding would have forced the DB to treat input as data only. ORM is a library or framework which allows developers to use familiar programming language objects to query and manipulate data, instead of writing raw SQL.

Insufficient server-side input validation (no allowlists)

Input accepted from the user should be constrained (e.g., patient ID should be digits only; last name should match a conservative character allowlist). An allowlist in SQL is a security measure that explicitly defines a set of permitted actions and implicitly denies all others. Validation would not replace parameterization entirely, but it would reduce the risk of exploitability.

Over-privileged application DB account

The portal's DB user likely had broad "SELECT" over entire patient tables. With proper least-privilege (row-/column-level security or views exposing only necessary columns), successful SQLi would have yielded far less PII.

Why did existing defenses not catch it?

The organization likely had no Web Application Firewall (WAF) in front of the patient portal or the WAF was somehow misconfigured incorrectly missing the attack. Because the attack had appeared in a normal HTTPS session and looked like ordinary search traffic, signature-based rules did not trigger, and no positive-security (allowlist) policy existed to constrain acceptable user input.

Part B – The Playbook

What is an Incident Response Playbook?

A cybersecurity playbook is a comprehensive written set of guidelines that specify how security teams should respond to specific types of threats or incidents. It serves as an extensive framework or "recipe book" for cybersecurity experts, offering decision trees, standardized workflows, and step-by-step instructions customized to unique architecture and requirements of a business. The playbook guarantees that incident response is reliable, effective, and based on proven strategies (Tubin, 2024; Vandenberg, 2025).

By standardizing responses and defining roles and responsibilities, the playbook helps security teams navigate the chaos of cyber incidents as a strategic roadmap. By offering precise, feasible actions, it speeds up event detection, classification, and recovery. By lowering downtime and data loss, it improves operational efficiency, lowers human error, and supports business

continuity. Additionally, playbooks help firms keep ahead of changing cyberthreats by facilitating improvements through frequent updates and exercises (DigitalXRaid, 2024; Tubin, 2024; Vandenberg, 2025).

Overall, a cybersecurity playbook is essential for ensuring a swift, coordinated, and effective response to cyber incidents, safeguarding organizational assets and reputation.

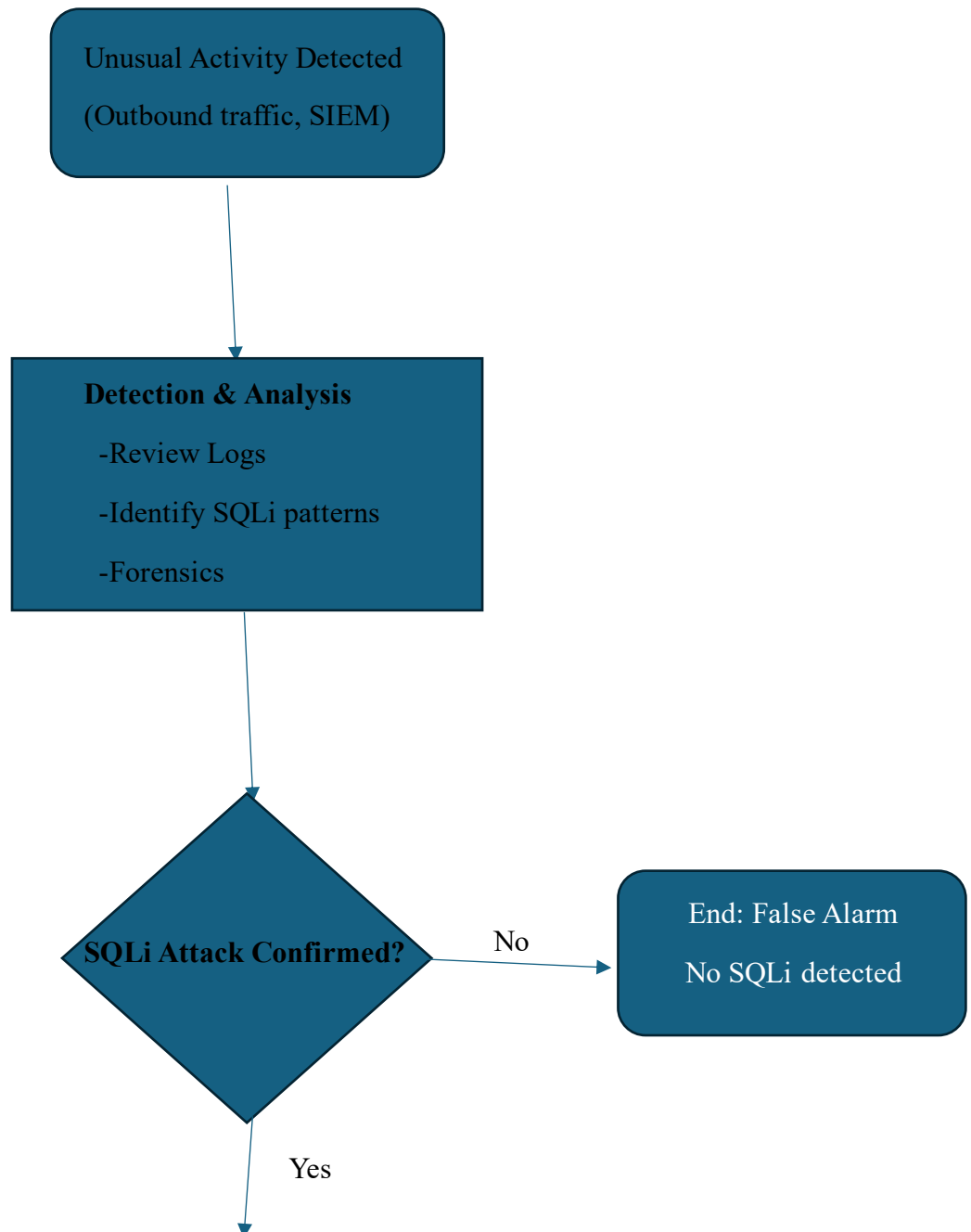
Incident Response Steps (SQLi Attack Playbook)

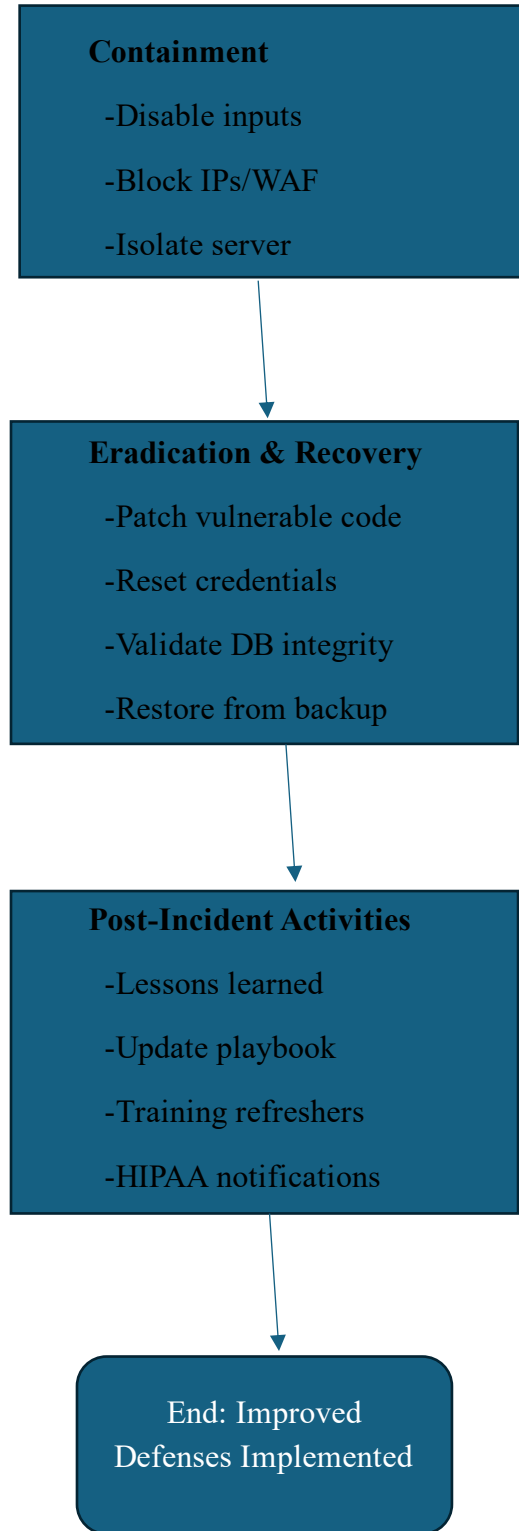
Step	Purpose	Scenario-Specific Actions	Sources
1. Preparation	Build readiness to handle SQLi attacks effectively.	➤ Develop SQLi Incident Response Playbook ➤ Enforce secure coding practices (ORMs, parameterized queries) ➤ Regular penetration tests & scans (SQLmap, Burp Suite) ➤ Deploy Web Application Firewall (WAF) with SQLi rules. ➤ Enable logging & SIEM monitoring of queries. ➤ Train developers/IT staff on SQLi awareness ➤ Test and maintain data backup/recovery plans	OWASP SQLi Prevention Cheat Sheet ; NIST SP 800-61r2
2. Detection & Analysis	Quickly identify SQLi attempts and confirm impact.	➤ Monitor outbound traffic via IDS/IPS ➤ Review web server logs for malicious patterns. ➤ Use database activity monitoring to detect abnormal queries. ➤ Correlate SIEM alerts with logs to confirm exfiltration. ➤ Conduct forensic analysis: entry point, timeline, affected tables. ➤ Notify compliance/legal (HIPAA review)	OWASP Logging Cheat Sheet ; HIPAA Journal (Detection Best Practices)
3. Containment	Limit further exploitation and stop data leakage.	➤ Disable vulnerable portal modules/inputs. ➤ Block attacker Ips in firewall/WAF. ➤ Isolate compromised server if exfiltration is active. ➤ Apply temporary WAF filters for SQLi payloads. ➤ Restrict database account privileges. ➤ Inform stakeholders of temporary service impact	SANS Incident Handler's Handbook ; NIST SP 800-61r2
4. Eradication & Recovery	Remove threat and restore secure operations.	➤ Patch vulnerable code (input validation, parameterized queries) ➤ Remove malicious backdoors/scripts. ➤ Run database integrity checks. ➤ Reset database/admin credentials. ➤ Restore from good backups if necessary. ➤ Conduct post-recovery penetration tests. ➤ Gradually restore services with monitoring enabled	OWASP SQLi Prevention Cheat Sheet ; NIST CSF v2.0 (Respond/Recover)
5. Post-Incident Activities	Learn from the incident and strengthen defenses.	➤ Hold post-mortem/root cause analysis. ➤ Update SQLi playbook with lessons learned. ➤ Conduct refresher training for developers/admins.	NIST SP 800-61r2 (Post Incident)

		➤ Submit HIPPA breach notifications (>5000 PII records) ➤ Perform risk assessment on apps/frameworks. ➤ Schedule ongoing red-team SQLi tests ➤ Track metrics (detection/containment/eradication times)	HIPPA Breach Notification Rule (45 CFR §§ 164.400-414)
--	--	--	--

Flowchart Diagram

The flowchart will visually represent some key incident response steps above. The flowchart will clearly display the progression of actions, from the initial identification of a SQLi attack through to post-incident activities.





Tools & Resources

Web Application Firewall (WAF)

This filters and monitors HTTP traffic between web applications and the internet. It can block malicious SQL injection payloads by analyzing request patterns. Personnel like Security Operations Center (SOC) Analysts and Web Administrators are responsible.

SOC Analysts will monitor WAF alerts for anomalies and configure rules to detect SQLi attempts. Web Administrators will implement custom security rules (e.g., input validation policies) and coordinate with developers for application fixes.

Intrusion Detection & Prevention System (IDS/IPS)

Detects suspicious network activity (e.g., unusual outbound queries from the patient portal server) and blocks connections if caught by the IPS. This helps confirm SQLi exploitation attempts. Responsible personnel include Network Security Engineers and Incident Response Teams (IRT).

Network Security Engineers will deploy IDS/IPS signatures for SQLi, monitor traffic logs, and fine-tune detection thresholds. An Incident Response Team will investigate alerts, correlate findings with logs, and initiates containment.

Database Activity Monitoring (DAM) Tool

This monitors queries sent to databases in real time. Can detect unusual query patterns (e.g., “UNION,” “SELECT” abuse, mass extraction PII) and prevent unauthorized access. Database Administrators (DBAs) and an IT Security Team will both be responsible for this tool.

DBAs can use the DAM to track database queries, block abnormal activity, and analyze affected tables. An IT Security Team can cross-check DAM logs with SIEM for forensic investigation and ensure compliance reporting (HIPAA).

Security Information and Event Management (SIEM) System

The SIEM will collect and correlate logs from WAF, IDS/IPS, DAM, and servers. This provides centralized visibility for detecting, analyzing, and responding to SQLi incidents.

SOC Analysts are responsible for this tool as they can configure correlation rules (e.g., flagging abnormal query entries from a single user). Incident Response Managers are also responsible as they can use SIEM results for decision-making and reporting to executives or regulators.

Endpoint Detection & Response (EDR) Solution

This resource protects servers hosting the patient portal by monitoring processes and file system changes. It can detect persistence mechanisms or malware dropped by SQLi exploitation. Responsible personnel would include System Administrators, which are tasked with ensuring agents are deployed and updated on all endpoints.

Lessons Learned

Employee Awareness and Training

Implement regular secure coding and cybersecurity awareness training for developers, IT staff, and end-users. The usage of parameterized queries, secure coding practices, and appropriate user input sanitization will all be covered for developers.

IT professionals will be more proficient at spotting abnormal activities, such as odd traffic patterns or SQL queries. Personnel who receive regular phishing and awareness training are also better at reporting suspicious activity sooner.

Technical Defenses

Deploy a Web Application Firewall (WAF) with SQLi detection and blocking capabilities. Before incoming requests ever make it to the backend database of the patient portal,

a WAF can inspect them for malicious SQL patterns and stop them. When secure programming is lacking, this serves as an extra line of defense.

Another action could be enforcing parameterized queries and stored procedures throughout the application code. Parameterized queries prevent user input from being interpreted as executable SQL commands, closing the main way being exploited in SQLi.

Incident Response Improvements

Conduct regular exercises and red-team simulations focused on SQLi and web application attacks. The incident response team can enhance containment efforts, coordination, and detection speed by practicing realistic attack scenarios. Additionally, it confirms the playbook's effectiveness and guarantees that professionals are aware of their responsibilities in the event of an emergency.

Compliance and Monitoring

Strengthen database logging and monitoring for abnormal query patterns. Continuous monitoring helps detect large-scale data exfiltration or repeated SQL injection attempts early. Logs also aid forensic investigations and compliance reporting (HIPAA requires audit trails for healthcare data access).

Policy and Governance

Establish a secure software development lifecycle (SDLC) policy. It is possible to identify vulnerabilities like SQLi earlier, before systems go live, by including security into every stage of development (requirements, design, coding, testing , and deployment and maintenance).

Together, these measures strengthen people, process, and technology defenses, reducing the likelihood of another SQLi attack and ensuring faster, more effective response if one occurs again.

References

- Adler, S. (2024, August 22). *CISA & Partners Issue Guidance & Best Practices for Event Logging and Threat Detection*. The HIPPA Journal.
<https://www.hipaajournal.com/guidance-best-practices-for-event-logging-threat-detection>
- Amazon AWS. (2024). *What is SQL? - SQL - AWS*. Amazon Web Services, Inc.
<https://aws.amazon.com/what-is/sql/>
- Cichonski, P., Millar, T., Grance, T., & Scarfone, K. (2012, August 6). *Computer Security Incident Handling Guide*. Csrc.nist.gov. <https://csrc.nist.gov/pubs/sp/800/61/r2/final>
- DigitalXRaid. (2024, November 25). *DigitalXRAID*. DigitalXRAID.
<https://www.digitalxraid.com/cyber-incident-playbooks-importance>
- Hofesh, B. (2022, April 8). *SQL Injection Attack: Real Life Attacks and Code Examples*. Bright Security. <https://brightsec.com/blog/sql-injection-attack>
- Kral, P. (2012, February 21). *Incident Handler's Handbook | SANS Institute*. Wwww.sans.org.
<https://www.sans.org/white-papers/33901>
- NIST. (2024). The NIST cybersecurity framework (CSF) 2.0. *The NIST Cybersecurity Framework (CSF) 2.0, 2.0(29)*. <https://doi.org/10.6028/nist.cswp.29>
- OWASP. (2024a). *Logging - OWASP Cheat Sheet Series*. Cheatsheetseries.owasp.org.
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- OWASP. (2024b). *SQL Injection*. OWASP. https://owasp.org/www-community/attacks/SQL_Injection

OWASP. (2025). *SQL Injection Prevention · OWASP Cheat Sheet Series*. Owasp.org; Owasp.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

SentinelOne. (2024, December 4). *What is SQL Injection? Examples & Prevention*. SentinelOne.

<https://www.sentinelone.com/cybersecurity-101/cybersecurity/sql-injection>

Tubin, G. (2024, July 15). *Why Cybersecurity Playbooks are Critical and 7 Steps to Create Your Own*. All-In-One Cybersecurity Platform - Cynet.

<https://www.cynet.com/cybersecurity/why-cybersecurity-playbooks-are-critical-and-7-steps-to-create-your-own>

U.S. Department of Health and Human Services. (2013, July 26). *Breach Notification Rule*. U.S.

Department of Health and Human Services. <https://www.hhs.gov/hipaa/for-professionals/breach-notification/index.html>

Vandenberg, P. (2025, July 18). *What Are Playbooks in Cybersecurity and How Do They Aid in*

Incident Management? | Cyware. Cyware. <https://www.cyware.com/blog/what-are-playbooks-in-cybersecurity-and-how-do-they-aid-in-incident>

VanMSFT. (2024, November 22). *SQL injection - SQL Server*. Microsoft.com.

<https://learn.microsoft.com/en-us/sql/relational-databases/security/sql-injection?view=sql-server-ver17>

w3schools. (2019). *SQL Introduction*. W3schools.com.

https://www.w3schools.com/sql/sql_intro.asp